CAPACITA iRede

TAREFA

Unidade 5 | Capítulo 1

Desenvolvimento do Projeto (Intermediário)

Prof.: Alberson Dantas

Unidade 4

Capítulo 2 | Objetivo 3

Enunciado:
Com o desenvolvimento do projeto os alunos colocam em prática todos os conteúdos abordados no módulo intermediário, aplicando de forma autônoma os conhecimentos adquiridos sobre Generics, interface gráfica - JavaFX, Banco de Dado com JDBC e Testes de Software com JUnit.

Instruções:

A atividade deverá ser desenvolvida individualmente ou conforme a organização definida para a turma, retomando o projeto construído até o momento para realizar uma evolução e aprimoramento da solução. Caso o aluno tenha optado por desenvolver um outro projeto, deverá alinhar previamente com o professor os detalhes desta atividade. Esta será uma entrega única referente a este módulo.

Unidade 5

Capítulo 1

Projeto 1: TaskManager – Gerenciador de Tarefas (Versão Console)

Objetivo do Projeto (Intermediário):

O objetivo nesta etapa é evoluir o TaskManager de uma aplicação de console para uma aplicação visual (GUI) utilizando JavaFX. Os dados das tarefas serão persistidos em um banco de dados relacional por meio de JDBC. Também será aplicada uma estrutura de código genérica com Generics, além de testes automatizados com JUnit.

Esse upgrade consolida a aplicação como um sistema real de tarefas, estruturado, testável, persistente e com interface amigável ao usuário.

Descrição Geral:

O sistema TaskManager permitirá ao usuário realizar operações básicas de controle de tarefas.

Novas Tecnologias e Conceitos Utilizados:

1. Generics

Criação de classes e métodos genéricos para manipulação de listas de tarefas. Uso de ?, ? extends T e ? super T para maior flexibilidade e reaproveitamento de código.

Exemplo prático:

```
public class ListaGenerica<T> {
    private List<T> elementos = new ArrayList<>();

    public void adicionar(T elemento) { ... }
    public void listarTodos() { ... }
    public void remover(T elemento) { ... }
}
```

2. JavaFX – Interface Gráfica

Estrutura básica:

- Criação da classe MainApplication que estende Application
- Implementação do método start(Stage stage)
- Troca da interação por terminal para uma interface visual.

Telas:

- Tela Principal (lista de tarefas com opção de marcar, remover ou editar)
- Tela de Cadastro/Edição de Tarefas
- Confirmação ao remover

Componentes JavaFX:

- Label, TextField, TextArea, Button, TableView, CheckBox
- Manipulação de eventos com setOnAction()
- Uso de VBox, HBox, GridPane, BorderPane para layout
- Separação entre lógica e visual com arquivos .fxml
- Estilização com arquivos .css

3. FXML e Scene Builder

- Criação das telas em .fxml, facilitando a separação de responsabilidades.
- Uso do Scene Builder para criação visual das interfaces.

Exemplo:

```
<Button text="Salvar" onAction="#salvarTarefa"/>
```

Lógica associada via Controller.java com anotações @FXML.

4. Acesso a Banco de Dados com JDBC

Configurações:

- Uso de um banco relacional (como SQLite ou MySQL)
- Criação de uma base de dados com tabela tarefas:

```
CREATE TABLE tarefas (
  id INTEGER PRIMARY KEY AUTO_INCREMENT,
  titulo VARCHAR(100),
  descricao TEXT,
  concluida BOOLEAN
);
```

Operações:

- Conectar-se ao banco com DriverManager
- Inserir, consultar, atualizar e excluir tarefas com PreparedStatement e ResultSet
- Aplicar transações com commit() e rollback() quando necessário
- Evitar SQL Injection com PreparedStatement

Arquitetura:

- Criação de classe TarefaDAO para isolar a persistência
- Conexão gerenciada via Conexao.java

5. Testes de Software com JUnit

Objetivo:

- Verificar a lógica do sistema de forma automatizada.
- Estimular o uso de TDD (Test-Driven Development)

Testes criados:

- Testar criação de tarefa
- Testar conclusão de tarefa
- Testar DAO com SQLite em memória
- Validação de dados

Estrutura:

- Uso das anotações @Test, @BeforeEach, @AfterEach
- Testes organizados em pacotes separados

Projeto 2: Finanças Pessoais (Versão Console)

Nome do Projeto: FinTrack

Descrição do Projeto (Versão Inicial):

O FinTrack é um sistema de controle de finanças pessoais via console, permitindo ao usuário:

- Cadastrar entradas (receitas) e saídas (despesas);
- Listar todas as transações;
- Exibir saldo atual;
- Remover uma transação.

Novas Tecnologias e Conceitos Utilizados

1. Generics

- Criação de classes genéricas para listas de registros e serviços.
- Métodos reutilizáveis para adicionar, remover e listar elementos com tipo genérico.
- Uso de curingas: ?, ? extends T, ? super T.

2. JavaFX – Interface Gráfica

Estrutura:

- FinApp estende Application e inicia o JavaFX;
- Utilização do método start(Stage) para exibir a tela inicial.

Telas:

- Tela principal com tabela de transações;
- Tela de nova transação (receita ou despesa);
- Tela de relatório/saldo.

Componentes utilizados:

- Label, TextField, TextArea, Button, DatePicker, CheckBox;
- TableView para listar transações com colunas (data, descrição, valor, tipo);
- VBox, HBox, GridPane, BorderPane para layout;
- Eventos com setOnAction().

3. FXML + Scene Builder + CSS

- Criação das telas com arquivos .fxml separados;
- Uso do Scene Builder para design visual;
- Lógica separada em controllers vinculados por fx:controller;
- Estilização com .css para aparência agradável.

4. Acesso a Banco de Dados com JDBC

Configuração:

Criação da base de dados com tabela transacoes:

```
CREATE TABLE transacoes (
  id INT PRIMARY KEY AUTO_INCREMENT,
  descricao VARCHAR(100),
  valor DECIMAL(10,2),
  tipo VARCHAR(10),
  data DATE
);
```

Operações:

- CRUD completo com PreparedStatement e ResultSet;
- Conexão segura com driver JDBC;
- DAO: TransacaoDAO para abstrair persistência;
- Conexao.java para centralizar acesso ao banco;
- Tratamento de erros SQL com try/catch e logs;
- Controle de transações com commit() e rollback() (opcional).

5. Testes de Software com JUnit

- Testes unitários para classes Transacao, RepositorioGenerico e TransacaoDAO;
- Utilização de SQLite em memória para testes de DAO;
- Anotações @BeforeEach, @AfterEach, @Test;
- Validação de cálculos de saldo, inserção e remoção de transações;
- Uso de Assertions.assertEquals(), assertThrows().

Projeto 3: Agenda de Contatos (Versão Console)

Nome do Projeto: MyContacts

Descrição do Projeto (Versão Inicial):

O MyContacts é um sistema simples de agenda de contatos no console. O usuário poderá:

- Adicionar contatos
- Listar contatos
- Buscar contatos por nome
- Remover contatos

O objetivo desta fase é transformar a agenda de contatos de console em uma aplicação visual (JavaFX) com persistência de dados em banco de dados relacional (via JDBC), uso de Generics para listas e buscas, e a aplicação de testes automatizados com JUnit.

Novas Tecnologias e Conceitos Utilizados

1. Generics

- Uso de uma estrutura de repositório genérica para manipular contatos de qualquer tipo (simples ou comercial).
- Métodos genéricos para buscar, listar e remover contatos.
- Aplicação de curingas (?, ? extends T, ? super T) para permitir flexibilidade de subclasses.

2. JavaFX – Interface Gráfica

Estrutura da aplicação:

- Classe AgendaApp estende Application e inicia a aplicação JavaFX.
- Método start(Stage) configura e exibe a interface principal.

Telas:

- Tela Principal: mostra todos os contatos com opções para buscar, editar e remover.
- Tela de Cadastro: formulário com campos de entrada e validação.
- Tela de Detalhes (opcional): exibe dados completos do contato.

Componentes:

- Label, TextField, Button, TableView, ComboBox;
- VBox, HBox, BorderPane, GridPane para layout;
- Manipulação de eventos com setOnAction().

3. FXML + Scene Builder + CSS

- Interface criada com arquivos .fxml para separar lógica da visualização.
- Scene Builder usado para montar telas visualmente.
- Estilização via .css.

4. Banco de Dados Relacional com JDBC

Configuração:

Uso de SQLite ou MySQL com tabela:

```
CREATE TABLE contatos (
    id INTEGER PRIMARY KEY AUTO_INCREMENT,
    nome VARCHAR(100),
    telefone VARCHAR(20),
    email VARCHAR(100),
    empresa VARCHAR(100) -- pode ser nulo
);
```

Funcionalidades:

- CRUD completo usando PreparedStatement e ResultSet;
- DAO: ContatoDAO para persistência;
- Conexão centralizada com Conexao.java;
- Proteção contra SQL Injection;

- Separação entre lógica da aplicação e persistência (DAO Pattern).

5. Testes com JUnit

O que testar:

- Inserção, remoção e busca de contatos;
- Validação de e-mails com ValidadorEmail;
- Testes de DAO com SQLite em memória.

Recursos JUnit:

- Anotações @BeforeEach, @AfterEach, @Test;
- Uso de Assertions;
- Testes independentes e automatizados.

Outras Instruções:

Para o desenvolvimento do projeto a turma deve se dividir nas mesmas equipes, pois se trata de uma continuação do projeto do módulo anterior. Devem desenvolver a próxima parte do projeto de acordo com o escolhido no módulo inicial. um dos projeto abaixo e seguir todas as instruções abaixo para a depender do projeto escolhido.

Caso o aluno tenha optado por desenvolver um outro projeto atender os mesmos critérios:

- Usar conceitos de Generics;
- Aplicar Interface Gráfica – JavaFX;
- Utilizar Banco de Dados com JDBC;
- Utilizar JUnit para realizar testes de softwares, isso deve ser acordado com o professor e tutores.

Essa é a 2° (segunda) entrega, haverá mais 1 (uma) para o módulo avançado.

CAPACITA iRede

RESIDÊNCIA EM TIC 20

EXECUTOR iRede

INICIATIVA MCTI FUTURO

COORDENADORA PPI Softex

MINISTÉRIO DA CIÊNCIA, TECNOLOGIA E INOVAÇÃO

GOVERNO FEDERAL BRASIL UNIÃO E RECONSTRUÇÃO

*Este projeto foi apoiado pelo Ministério da Ciência, Tecnologia e Inovações, com recursos da Lei nº 8.248, de 23 de outubro de 1991, no âmbito do PPI-SOFTEX, coordenado pela Softex e publicado Residência em TIC 20, DOU 01245.002631/2023-58.